ESCOLA SECUNDÁRIA DA BOA NOVA

Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos

StickMania – Jogo de Hóquei em Patins 2D

Rafael da Hora Silveira

junho de 2027



ESCOLA SECUNDÁRIA DA BOA NOVA

Curso Profissional de Técnico/a de Desenvolvimento de Software

StickMania – Jogo de Hóquei em Patins 2D

Sob o acompanhamento do Dr. Fernando Filipe

Rafael da Hora Silveira e Vasco Andreia

junho de 2027

Agradecimentos

Agradecemos ao nosso orientador, Dr. Fernando Filipe, e à professora Cristina Paes pelo acompanhamento, pelas sugestões e pela disponibilidade ao longo do projeto. Agradecemos também aos professores do curso, aos colegas que testaram o jogo e deram opiniões, e às nossas famílias pelo apoio.

Resumo

Este relatório descreve o StickMania, um jogo de computador 2D de hóquei em patins desenvolvido em Python com a biblioteca pygame-ce, no âmbito da Prova de Aptidão Profissional. O jogador controla uma equipa contra o computador, com as oito equipas portuguesas, os seus símbolos e equipamentos. O jogo tem física da bola com atrito e ressaltos, colisões entre jogadores, faltas, guarda-redes, uma inteligência artificial com papéis táticos e três níveis de dificuldade, menus completos e uma câmara em perspetiva que mostra o rinque como numa transmissão de televisão, e um modo carreira em que o jogador escolhe um clube e joga ou simula um campeonato contra todas as outras equipas, guardado numa base de dados SQLite, com os planteis reais dos clubes e a tabela dos melhores marcadores. Os sons (apito, tacadas, público) são criados por síntese no próprio código. O desenvolvimento foi iterativo, com versões jogáveis sucessivas e controlo de versões no GitHub. O relatório apresenta a análise de requisitos, a modelação (casos de uso, arquitetura, classes, estados, fluxogramas e modelo de dados), a implementação dos algoritmos principais, os testes realizados e a gestão do projeto.

Índice

1. Introdução 10
1.1 Enquadramento e contextualização do tema 10
1.2 Motivação e pertinência do projeto 10
1.3 Objetivos (geral e específicos) 10
1.4 Metodologia de trabalho (processo adotado; planeamento) ◆ MADS 11
1.5 Estrutura do relatório 12
2. Análise do sistema 12
2.1 Descrição do problema e da necessidade a resolver ◆ MADS 12
2.2 Intervenientes e contexto de utilização ◆ MADS 12
2.3 Levantamento e análise de requisitos ◆ MADS 13
2.3.1 Técnicas de levantamento utilizadas ◆ MADS 13
2.3.2 Requisitos funcionais (o que o sistema deve fazer) ◆ MADS 13
2.3.3 Requisitos não funcionais (segurança, autonomia, robustez) ◆ MADS 14
2.4 Modelação do sistema ◆ MADS 14
2.4.1 Cenários de utilização / casos de uso ◆ MADS 14
2.4.2 Arquitetura do sistema (diagrama de blocos: sensores → controlo → atuadores) ◆ MADS 16
2.4.3 Modelação comportamental (fluxogramas / máquina de estados) ◆ MADS 18
2.5 Projeto do hardware 21
2.5.1 Componentes: sensores, atuadores e controlador 21
2.5.2 Esquema elétrico e ligações 22
2.5.3 Estrutura mecânica 22
2.6 Prototipagem (protótipo físico e/ou simulação) ◆ MADS 22
3. Implementação 23
3.1 Montagem do hardware 23
3.2 Programação do controlo (lógica e algoritmos) 23
3.3 Integração hardware – software 32
3.4 Decisões técnicas e dificuldades ultrapassadas 32
4. Testes e validação 34
4.1 Plano de testes 34
4.2 Ensaios, calibração e medições 36
4.3 Resultados obtidos e ajustes 38
5. Gestão do projeto 38
5.1 Planeamento e cronograma ◆ MADS 38
5.2 Gestão de riscos ◆ MADS 39

5.3 Controlo de versões e trabalho em equipa (Git/GitHub) 40
6. Conclusão 41
7. Anexos 42

Índice de figuras e tabelas

Figura 1 – Diagrama de casos de uso
Figura 2 – Arquitetura do sistema em blocos (entrada → controlo → saída)
Figura 3 – Diagrama de classes (simplificado)
Figura 4 – Modelo de dados da base de dados SQLite do modo carreira
Figura 5 – Máquina de estados do jogo
Figura 6 – Fluxograma do ciclo principal do jogo
Figura 7 – Fluxograma de decisão da inteligência artificial
Figura 8 – Fluxograma do modo carreira
Figura 9 – Captura de ecrã do StickMania durante uma partida
Figura 10 – Modo carreira: escolha da equipa e da dificuldade
Figura 11 – Modo carreira: classificação com a forma, próximo jogo com as probabilidades e resultados da jornada
Figura 12 – Modo carreira: resultado de um jogo simulado
Figura 13 – Modo carreira: fim da época, com o campeão e os melhores marcadores
Figura 14 – Remate na primeira pessoa: mira, círculo de erro e barra de força
Figura 15 – Estilo arcade: festa de um golo, com o nome de quem marcou
Figura 16 – Menu principal novo, com a engrenagem das definições no canto

Tabela 1 – Descrição do caso de uso «Jogar partida»
Tabela 2 – Requisitos de hardware e software para correr o jogo
Tabela 3 – Parâmetros dos níveis de dificuldade da IA
Tabela 4 – Controlos do jogo
Tabela 5 – Plano de testes
Tabela 6 – Medição da jogabilidade antes e depois das correções (média por jogo de 3 min, IA contra IA)
Tabela 7 – Bola e guarda-redes antes e depois das correções (média por jogo de 3 min)
Tabela 8 – Modelo de simulação dos jogos: antigo e novo (600 épocas, 16 800 jogos)
Tabela 9 – Remate na primeira pessoa: percentagem de golos no nível normal (força 65 %)
Tabela 10 – Cronograma do projeto
Tabela 11 – Gestão de riscos

Lista de siglas e abreviaturas

2D – Duas dimensões

FPS – Frames (imagens) por segundo

GR – Guarda-redes

HUD – Heads-Up Display – informação mostrada por cima do jogo (marcador, mini-mapa)

IA – Inteligência Artificial

PAP – Prova de Aptidão Profissional

PNG – Portable Network Graphics – formato de imagem

SQL – Structured Query Language – linguagem de bases de dados

UML – Unified Modeling Language – linguagem de diagramas de software

WAV – Waveform Audio File – formato de ficheiro de som

1. Introdução

1.1 Enquadramento e contextualização do tema

O hóquei em patins é uma modalidade com grande tradição em Portugal. A seleção nacional é uma das mais vitoriosas do mundo e clubes como o FC Porto, o Benfica, o Sporting, o OC Barcelos ou a UD Oliveirense enchem pavilhões todas as épocas. Apesar disso, quase não existem videojogos atuais sobre a modalidade: há muitos jogos de futebol e de basquetebol, mas o hóquei em patins ficou esquecido.

O StickMania é um jogo de computador 2D de hóquei em patins, feito em Python com a biblioteca pygame-ce. O jogador controla uma equipa de cinco elementos (quatro jogadores de campo e um guarda-redes) contra uma equipa controlada pelo computador, num rinque visto de lado e de cima, como numa transmissão de televisão. O projeto foi desenvolvido no âmbito da Prova de Aptidão Profissional (PAP) do Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos.

1.2 Motivação e pertinência do projeto

A ideia nasceu do gosto pela modalidade e da vontade de ver um jogo com as equipas portuguesas. Ao mesmo tempo, um jogo é um projeto muito completo para aprender programação: obriga a tratar de física (movimento, atrito, colisões e ressaltos), inteligência artificial, interfaces gráficas, animação, organização do código em módulos e testes.

O projeto é pertinente porque junta quase todas as matérias do curso num produto que qualquer pessoa consegue experimentar, e porque ajuda a divulgar uma modalidade com pouca presença nos videojogos.

1.3 Objetivos (geral e específicos)

Objetivo geral: desenvolver um jogo de hóquei em patins 2D jogável, estável e fluido, em que um jogador humano defronta uma equipa controlada pelo computador.

Objetivos específicos:

- Simular a física da bola e dos jogadores (aceleração, atrito do piso, ressaltos nas tabelas e nas balizas).

- Criar uma inteligência artificial com papéis táticos (avançado, médio e defesa) e três níveis de dificuldade.
- Implementar as regras principais: duas partes com tempo, golos, faltas e reposição da bola ao centro.
- Criar menus completos: menu principal, seleção de equipas, definições (controlos e ecrã inteiro), pausa e ecrã final.
- Mostrar as oito equipas com símbolos e equipamentos com as cores de cada clube.
- Desenhar o rinque com uma câmara em perspetiva, bancadas com público e balizas em 3D.
- Criar um modo carreira: um campeonato contra todas as outras equipas, com classificação, guardado numa base de dados SQLite para se poder continuar mais tarde, e com a opção de simular os jogos com resultados realistas.
- Dar som ao jogo (apito, tacadas, tabelas, postes, golos e público), criado por síntese no código.
- Manter o código organizado em módulos e com controlo de versões no GitHub.

1.4 Metodologia de trabalho (processo adotado; planeamento) ◆ MADS

Foi seguido um processo de desenvolvimento iterativo e incremental. Em vez de fazer tudo de uma vez, o jogo foi construído em pequenas versões, cada uma jogável, que foram sendo melhoradas:

- **Versão 1 –** rinque visto de cima, jogadores a mexer e bola com física simples.
- **Versão 2 –** física da bola corrigida, colisões, guarda-redes e golos.
- **Versão 3 –** interface do mockup (marcador, mini-mapa, cartão do jogador) e símbolos dos clubes.
- **Versão 4 –** mira de remate com o rato, bola colada ao stick, níveis de dificuldade e faltas.
- **Versão 5 –** animações de patinagem, balizas desenhadas no código e câmara em perspetiva com bancadas.
- **Versão 6 –** correção das colisões, da bola e do guarda-redes, nova IA tática e modo carreira (campeonato guardado em SQLite).
- **Versão 7 –** menu principal novo com uma engrenagem para as definições, ecrã inteiro e gráficos melhorados (piso, luzes, sombras e linhas suaves).
- **Versão 8 –** sons, simulação de jogos com resultados realistas, força dos clubes também no jogo e ecrãs do modo carreira redesenhados.
- **Versão 9 –** planteis reais (4 jogadores de campo e 1 guarda-redes, com os nomes de cada clube), marcadores de cada golo e tabela dos melhores marcadores.

- **Versão 10 –** estilo arcade (golos com câmara lenta e confetes, impactos com pausa e tremer, textos de banda desenhada), que se liga e desliga nas Definições.
- **Versão 11 –** remate na primeira pessoa (apontar a qualquer ponto da baliza e escolher a força) e MODO TESTE para treinar remates contra o guarda-redes.

No fim de cada versão o jogo era testado e as falhas encontradas entravam na lista da versão seguinte. Todo o código está guardado no GitHub, o que permite ver a evolução e voltar atrás quando uma alteração não resulta (por exemplo, uma folha de sprites nova que acabou por ser revertida). O planeamento detalhado está no capítulo 5.

1.5 Estrutura do relatório

Depois desta introdução, o capítulo 2 faz a análise do sistema: o problema, os intervenientes, os requisitos e a modelação com diagramas (casos de uso, arquitetura, classes, estados e fluxogramas). O capítulo 3 descreve a implementação, os algoritmos principais e as dificuldades ultrapassadas. O capítulo 4 apresenta os testes realizados e os resultados. O capítulo 5 trata da gestão do projeto (cronograma, riscos e controlo de versões). Por fim, o capítulo 6 apresenta a conclusão e o capítulo 7 os anexos.

2. Análise do sistema

2.1 Descrição do problema e da necessidade a resolver ◆ MADS

Quem gosta de hóquei em patins praticamente não tem jogos de computador da modalidade. Os poucos que existem são antigos, não têm as equipas portuguesas ou não reproduzem bem o jogo: a bola rápida, os ressaltos nas tabelas, o stick e o jogo atrás da baliza.

A necessidade a resolver é ter um jogo simples de instalar, que funcione num computador normal sem internet, com as equipas portuguesas, e que seja divertido tanto para quem conhece a modalidade como para quem nunca a viu. Para isso o jogo tem de ter uma física credível, adversários controlados pelo computador que joguem de forma inteligente e uma apresentação parecida com a televisão.

2.2 Intervenientes e contexto de utilização ◆ MADS

Intervenientes:

- **Jogador:** a pessoa que usa o jogo. Escolhe as equipas e a dificuldade e controla um jogador de cada vez.
- **Computador (IA):** controla a equipa adversária, os colegas do jogador e os dois guarda-redes.
- **Equipa de desenvolvimento:** Rafael da Hora Silveira e Vasco Andreia, responsáveis pela análise, programação, testes e documentação.
- **Orientadores:** Dr. Fernando Filipe e professora Cristina Paes, que acompanharam e validaram o trabalho.
- **Júri da PAP:** avalia o produto final e este relatório.

Contexto de utilização: o jogo corre num computador com Windows, numa janela de 1280×720 píxeis, e usa teclado e rato. Não precisa de internet nem de criar conta. Arranca com um duplo clique no ficheiro jogar.bat, que procura o Python e instala o pygame-ce se for preciso.

2.3 Levantamento e análise de requisitos ◆ MADS

Os requisitos dizem o que o jogo tem de fazer e em que condições. Foram tirados do anteprojeto do Stick Mania, a partir do problema, dos objetivos e da organização do jogo. Há requisitos funcionais, que descrevem o que o sistema faz, e requisitos não funcionais, ligados à segurança, à autonomia e à robustez.

2.3.1 Técnicas de levantamento utilizadas ◆ MADS

Para chegar a estes requisitos usámos quatro formas.
Leitura do anteprojeto. Lemos o documento e apontámos o que o jogo tem de ter: física, inteligência artificial, menus, torneio e base de dados.
Análise da modalidade. Usámos as regras básicas do hóquei em patins: duas equipas, guarda-redes, bola, stick, golos, tempo e classificação.
Comparação com outros jogos. Há muitos jogos de futebol e de basquetebol, mas quase nenhum de hóquei em patins atual. Isso mostrou o que faltava.
Revisão em grupo. Rafael Hora Silveira e Vasco Andreia reviram a lista com a professora Cristina Paes. Ainda não foram feitos inquéritos nem entrevistas.

2.3.2 Requisitos funcionais (o que o sistema deve fazer) ◆ MADS

Estes requisitos descrevem o que o jogo deve fazer.
01. O jogo deve abrir uma janela e continuar a correr, lendo o teclado.
02. Deve ter menu principal, escolha de equipas, pausa, definições e ecrã de resultado.
03. Deve deixar pausar e continuar sem perder o marcador nem o tempo.
04. Deve contar o tempo, separar as duas partes, detetar o golo e repor a bola no centro.
05. Deve mostrar o nome ou o emblema das duas equipas.
06. Deve mostrar jogadores de campo e guarda-redes, com stick e bola.
07. Deve mudar a animação do jogador: parado, patinar, conduzir, passar, rematar e chocar.

08. Deve mover o jogador ativo em várias direções, com aceleração e um limite de velocidade.
09. Deve deixar trocar o jogador que está a ser controlado.
10. Deve simular o atrito do rinque de madeira.
11. Deve calcular o movimento da bola, os ressaltos, a condução no stick, o passe e o remate.
12. Deve detetar colisões entre jogadores, bola, balizas e limites do rinque.
13. A equipa do computador deve perseguir a bola, defender, passar e rematar.
14. O guarda-redes deve mover-se na baliza para tentar defender.
15. Durante o jogo deve aparecer o marcador, o tempo, a parte e a pausa.
16. Deve haver um mini-mapa com os dez jogadores e a bola, e os atributos do jogador ativo.
17. No torneio deve atualizar vitórias, empates, derrotas e golos.
18. Deve guardar equipas, jogadores, partidas, estatísticas e classificação em SQLite.
19. Deve dar 3 pontos pela vitória, 1 pelo empate e 0 pela derrota, e ordenar a classificação.

2.3.3 Requisitos não funcionais (segurança, autonomia, robustez) ◆ MADS

Estes requisitos descrevem a segurança, a autonomia e a robustez do jogo.
01. As consultas à base de dados devem ser parametrizadas, para ninguém enviar comandos pela interface.
02. Os dados do torneio ficam só no computador. Não é preciso criar conta nem enviar dados para fora.
03. O jogo não guarda dados pessoais sensíveis. Guarda equipas, jogadores, partidas e estatísticas.
04. O jogo funciona sem internet e sem servidor.
05. Dá para continuar o torneio mais tarde, no mesmo computador.
06. O jogo não deve fechar sozinho ao pausar, ao marcar um golo ou ao mudar de parte.
07. A bola e os jogadores não devem ficar presos fora do rinque.
08. Se a gravação falhar, o que já estava guardado não deve ser apagado.
09. Não pode haver resultados impossíveis, como golos negativos.
10. O jogo deve andar de forma fluida, perto de 60 imagens por segundo.
11. O marcador, o tempo e o mini-mapa devem estar visíveis durante a partida.
12. O código fica dividido por partes, para cada um mudar a sua parte sem estragar o resto.

2.4 Modelação do sistema ◆ MADS

2.4.1 Cenários de utilização / casos de uso ◆ MADS

O diagrama de casos de uso mostra o que cada ator pode fazer no sistema. O Jogador escolhe as equipas e a dificuldade, consulta as definições (controlos e ecrã inteiro), joga a partida, pausa e vê o resultado. Durante a partida pode mover o jogador, passar, rematar na primeira pessoa e trocar de jogador. O Computador controla a equipa adversária e os

guarda-redes. No modo carreira, o Jogador joga um campeonato completo e consulta a classificação e o calendário entre os jogos. No MODO TESTE treina remates sozinho contra o guarda-redes.

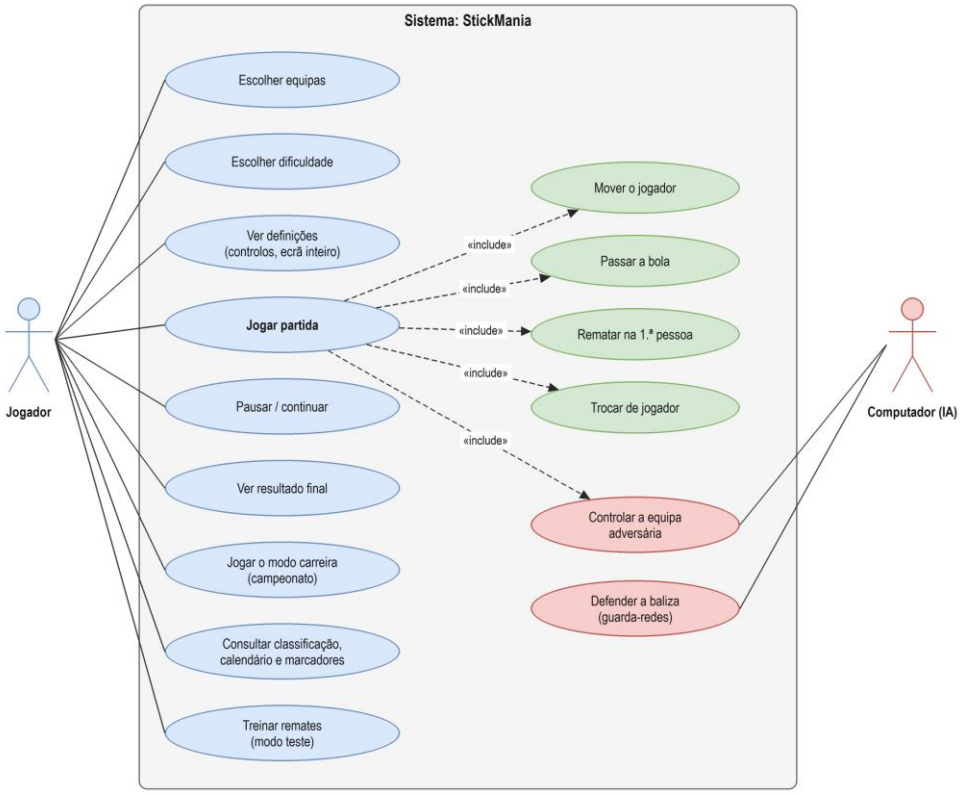


Figura 1 – Diagrama de casos de uso

Tabela 1 – Descrição do caso de uso «Jogar partida»

Campo	**Descrição**
Ator principal	Jogador (o Computador participa como ator secundário)
Pré-condição	As equipas e a dificuldade foram escolhidas no ecrã de seleção.
Fluxo principal	1. O sistema coloca os jogadores e a bola no centro. 2. O jogador move-se com WASD, passa com Z e remata com X (na primeira pessoa). 3. A IA controla os restantes jogadores. 4. Quando a bola entra na baliza, o golo é somado e a bola volta ao centro. 5. No fim da 1.ª parte há intervalo; no fim da 2.ª parte aparece o resultado final.
Fluxos alternativos	ESC ou P abre a pausa (continuar, reiniciar, definições, menu ou sair). Uma entrada irregular é assinalada como falta e a bola é reposta.
Pós-condição	O resultado final é mostrado; o jogador pode jogar de novo, mudar de equipas ou voltar ao menu.

2.4.2 Arquitetura do sistema (diagrama de blocos: sensores → controlo → atuadores) ◆ MADS

O modelo «sensores → controlo → atuadores» foi adaptado a um sistema de software: os «sensores» são o teclado e o rato, o «controlo» é o ciclo principal do jogo e os «atuadores» são o ecrã e o som. Por baixo ficam os dados que o jogo carrega ao arrancar. Cada bloco corresponde a um ou mais ficheiros Python, o que permite alterar uma parte sem estragar as outras.

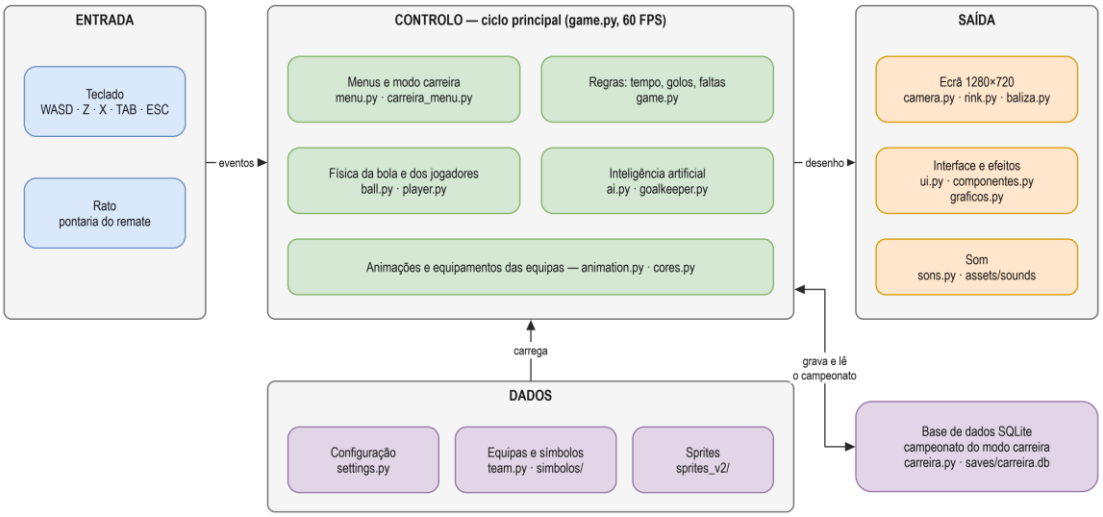


Figura 2 – Arquitetura do sistema em blocos (entrada → controlo → saída)

O diagrama de classes mostra as classes principais e como se ligam. A classe Game é o centro: tem o rinque, a bola, os oito jogadores de campo, os dois guarda-redes, os sete agentes de IA (todos os jogadores menos o humano), a interface e os menus. A Camera pertence ao Rink e é usada para desenhar tudo em perspetiva.

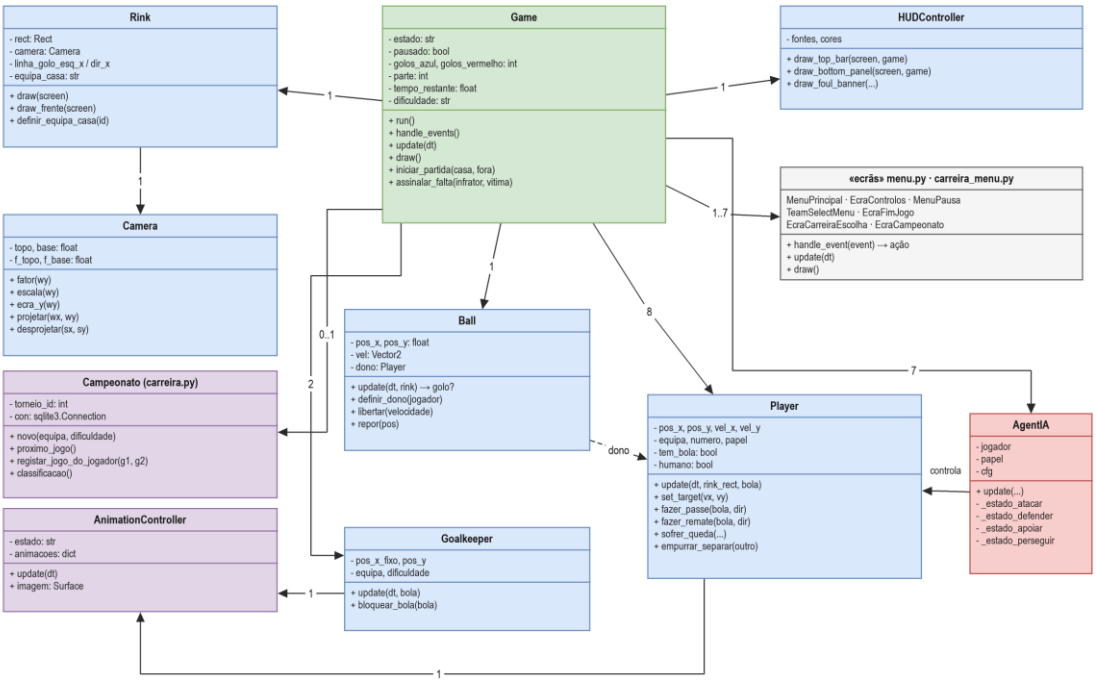


Figura 3 – Diagrama de classes (simplificado)

O modo carreira guarda o campeonato numa base de dados SQLite (ficheiro saves/carreira.db). O modelo de dados mostra as tabelas e as relações: cada torneio pertence à equipa escolhida pelo jogador e tem 28 partidas e uma linha de classificação por equipa; cada partida liga duas equipas (casa e fora). Cada equipa tem 5 jogadores (tabela Jogador) e a tabela Estatistica guarda quantos golos cada jogador marcou em cada partida, o que permite fazer a tabela dos melhores marcadores com uma consulta SQL.

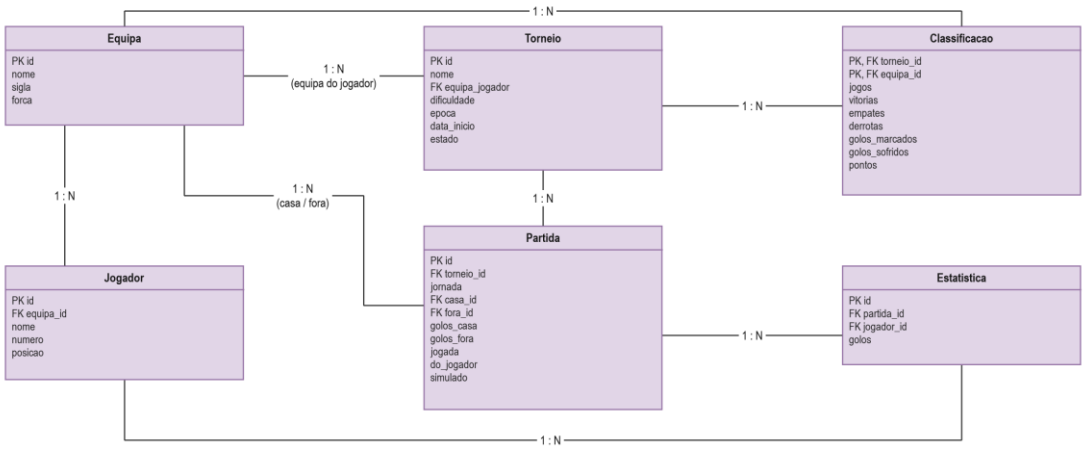


Figura 4 – Modelo de dados da base de dados SQLite do modo carreira

2.4.3 Modelação comportamental (fluxogramas / máquina de estados) ◆ MADS

O jogo funciona como uma máquina de estados: em cada momento está num estado (menu, seleção, partida, pausa, etc.) e as ações do jogador fazem-no mudar de estado. No código isto é a variável estado da classe Game, junto com as variáveis pausado e fim_jogo.

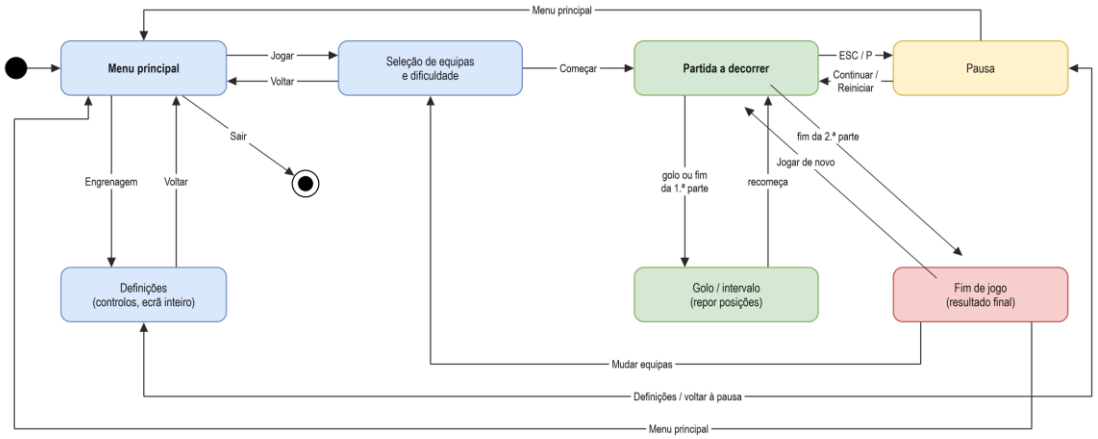


Figura 5 – Máquina de estados do jogo

O fluxograma seguinte mostra o ciclo principal, que se repete 60 vezes por segundo: ler o teclado e o rato, atualizar tudo e desenhar. A física e a lógica usam o tempo que passou desde o frame anterior (dt), para o jogo andar à mesma velocidade em computadores mais rápidos ou mais lentos.

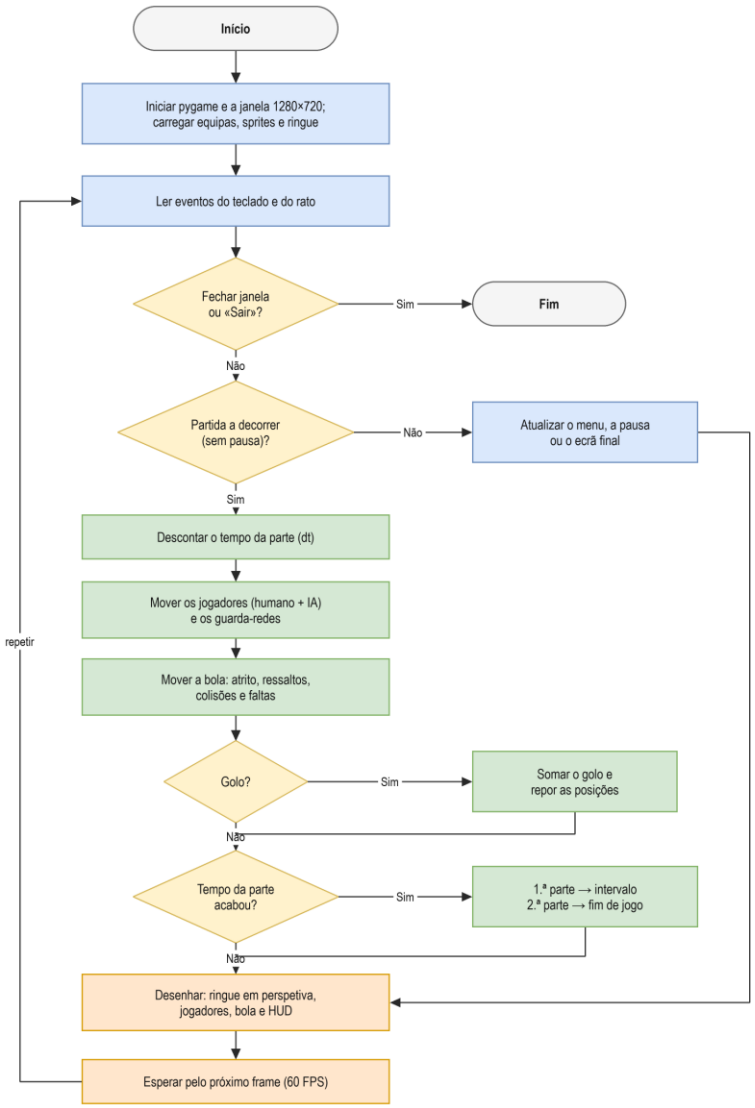


Figura 6 – Fluxograma do ciclo principal do jogo

Cada jogador da IA decide o que fazer em cada frame. Se tem a bola, ataca: contorna a baliza se estiver atrás dela, remata para o canto mais longe do guarda-redes se estiver perto e com a linha livre, passa a um colega desmarcado se estiver pressionado, ou conduz a bola fintando quem aparece à frente. Se não tem a bola e o adversário tem, os jogadores organizam-se numa fila: o mais perto do portador pressiona, o segundo faz a cobertura entre a bola e a baliza e os outros fecham os espaços. Quando é um colega a ter a bola, abrem espaço para receber. Com a bola solta, só vai buscá-la o jogador a quem o passe se destina ou o mais perto, e vai ao ponto onde a bola vai estar, não onde está.

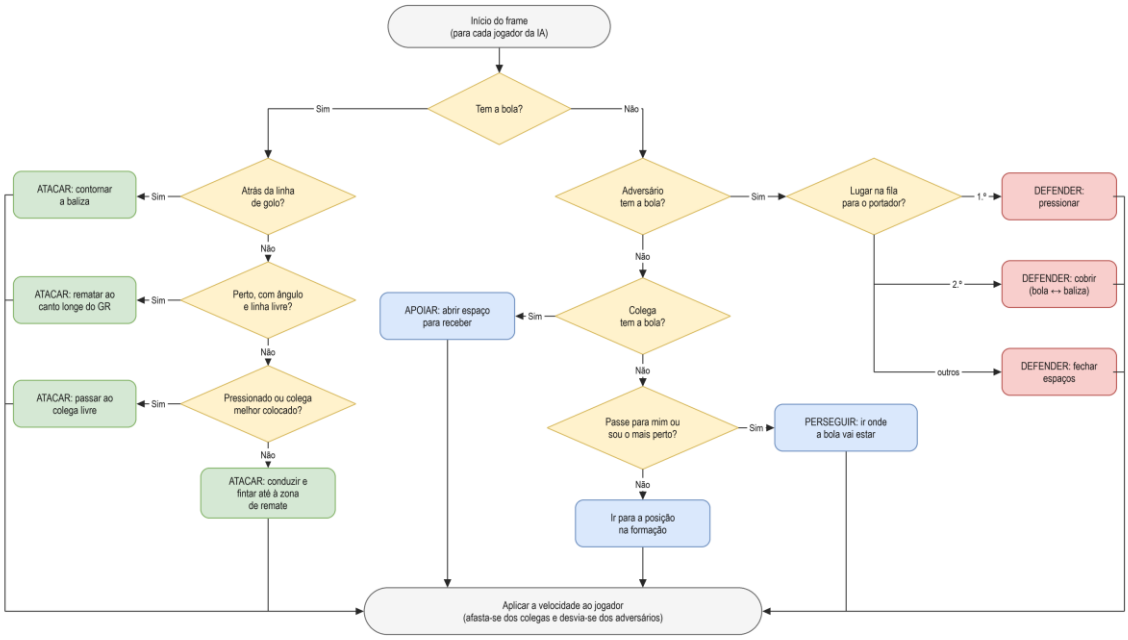


Figura 7 – Fluxograma de decisão da inteligência artificial

O modo carreira acrescenta dois estados ao jogo: a escolha da equipa (carreira_escolha) e o ecrã do campeonato (carreira). O fluxograma seguinte mostra como se passa de um para o outro e para as partidas.

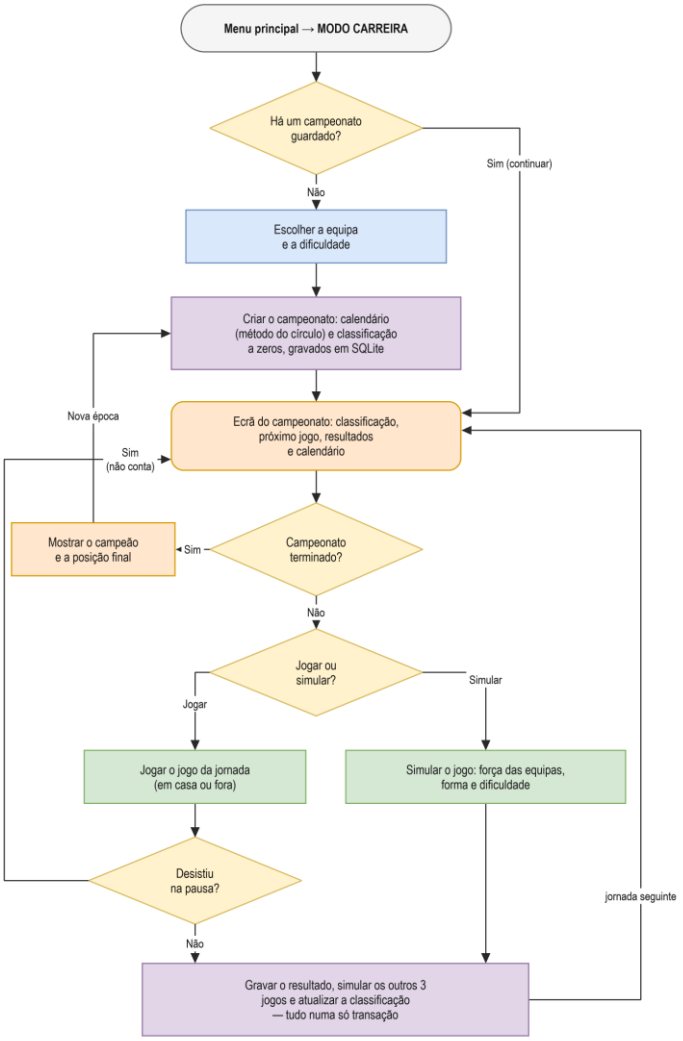


Figura 8 – Fluxograma do modo carreira

2.5 Projeto do hardware

O StickMania é um projeto só de software, por isso não há hardware a construir. Esta secção descreve o equipamento necessário para correr o jogo.

2.5.1 Componentes: sensores, atuadores e controlador

Não há sensores nem atuadores físicos. Os dispositivos usados são os de qualquer computador:

Tabela 2 – Requisitos de hardware e software para correr o jogo

Componente	Mínimo	Recomendado
Processador	Dual-core 2 GHz	Quad-core 2,5 GHz ou superior
Memória RAM	4 GB	8 GB

Componente	Mínimo	Recomendado
Ecrã	1280×720	1920×1080
Periféricos	Teclado	Teclado e rato
Sistema operativo	Windows 10	Windows 10 ou 11
Software	Python 3.10 + pygame-ce	Python 3.12 ou mais recente + pygame-ce

2.5.2 Esquema elétrico e ligações

Não se aplica: o projeto não tem circuitos elétricos.

2.5.3 Estrutura mecânica

Não se aplica: o projeto não tem estrutura mecânica.

2.6 Prototipagem (protótipo físico e/ou simulação) ◆ MADS

Em vez de um protótipo físico, foram feitos protótipos jogáveis do próprio jogo. O primeiro mostrava o rinque visto de cima, com jogadores e bola simples. As versões seguintes acrescentaram a física, a IA, a interface e, por fim, a câmara em perspetiva com bancadas, tabelas com altura e balizas em 3D.

Antes de programar a interface foi feito um mockup (desenho de como o ecrã devia ficar): marcador no topo, mini-mapa, cartão do jogador ativo e controlos em baixo. A figura seguinte mostra o estado atual do jogo.



Figura 9 – Captura de ecrã do StickMania durante uma partida

3. Implementação

3.1 Montagem do hardware

Não há hardware para montar. Em vez disso, foi preparado o ambiente de desenvolvimento:

- **Linguagem:** Python 3 (versão 3.10 ou mais recente).
- **Biblioteca gráfica:** pygame-ce 2.5, para a janela, o desenho, o teclado, o rato e o tempo.
- **Editor:** Visual Studio Code.
- **Controlo de versões:** Git e GitHub (repositório RafaelSilveira23/Stickmaniapap).
- **Diagramas:** draw.io (diagrams.net).
- **Arranque:** o ficheiro jogar.bat procura um Python compatível, instala o pygame-ce se faltar e abre o jogo.

3.2 Programação do controlo (lógica e algoritmos)

O código está dividido em módulos, um por responsabilidade (ver a arquitetura na secção 2.4.2). Seguem-se os algoritmos mais importantes.

Ciclo principal. game.py tem a classe Game, com o método run() que repete: handle_events() (ler o teclado e o rato), update(dt) (atualizar tudo) e draw() (desenhar). O pygame limita o ciclo a 60 FPS e dt é o tempo, em segundos, desde o frame anterior.

Física da bola. Em cada frame a bola anda vel × dt. O atrito do piso de madeira multiplica a velocidade por 0,991 quando a bola vai rápida e por 0,975 quando vai lenta (abaixo de 120 px/s), e abaixo de 8 px/s a bola para. Ao bater numa tabela, a componente da velocidade perpendicular à tabela troca de sinal e é multiplicada pelo coeficiente de ressalto 0,78. Um pequeno efeito lateral (spin) torna os ressaltos menos previsíveis. O passe tem força 400 px/s e o remate 650 px/s. Os cantos do rinque são arredondados (raio 50 px): se a bola passa a curva da tabela, volta a ser posta dentro dela e ressalta segundo a normal da curva.

Bola no stick. Quando um jogador tem a bola, ela fica a 22 px do centro, do lado para onde ele olha. Em vez de mudar de lado instantaneamente, o ângulo da bola à volta do jogador roda no máximo 0,24 rad por frame (cerca de 14 rad/s), como se o stick a arrastasse, e quando alguém apanha a bola ela desliza até ao stick a 9 px por frame. A bola no stick também nunca pode ficar dentro da rede da baliza.

Movimento e colisões dos jogadores. O jogador não muda de velocidade de repente: acelera até à velocidade pretendida (200 px/s, ou 260 px/s com Shift) com uma aceleração de 2000 px/s². As colisões entre jogadores são detetadas pela distância entre os centros (menos de 38 px). Quando chocam, acontecem três coisas: são separados (cada um anda metade da sobreposição); a velocidade com que vinham um contra o outro é anulada, como num choque inelástico, para não ficarem a empurrar-se e a tremer; e voltam a ser mantidos dentro do rinque e fora das balizas.

Desarmes. Quando um defensor choca com quem tem a bola, há uma tentativa de desarme com probabilidade p = 0,42 + (defesa do defensor − 0,5 × velocidade do portador − 30) / 100, limitada entre 15 % e 85 % (o jogador humano tem +18 %). Se ganha, a bola salta para o lado dele. Quem acabou de ganhar a bola fica 0,6 s protegido e quem a perdeu fica 0,8 s sem poder desarmar, o que acabou com o «pingue-pongue» da bola a mudar de dono várias vezes por segundo. Só há falta em choques fortes ou cargas por trás.

Inteligência artificial. Cada jogador da IA tem um papel (avançado, médio ou defesa) e uma posição na formação que acompanha a bola. Em cada frame escolhe um de quatro comportamentos (atacar, defender, apoiar ou perseguir), como mostra o fluxograma da secção 2.4.3. O movimento usa três forças somadas: ir para o alvo (abrandando ao chegar, para não passar e voltar para trás), afastar-se dos colegas que estão a menos de 62 px, e, com a bola, desviar-se dos adversários que estão à frente. Antes de passar ou rematar, a IA verifica se a linha está livre, calculando a distância de cada adversário ao segmento entre a bola e o alvo. Os passes são feitos para onde o colega vai estar (posição + velocidade × tempo da bola) e o colega que vai receber vai ao encontro da bola. Quando a posse muda, a IA demora um pouco a reagir (tempo de reação), como uma pessoa.

Tabela 3 – Parâmetros dos níveis de dificuldade da IA

Parâmetro	Fácil	Normal	Difícil
Velocidade normal (px/s)	140	175	215
Velocidade em sprint (px/s)	205	265	320
Raio para ir à bola (px)	135	190	265
Tempo entre remates (s)	2,2	1,4	0,75
Distância máxima de remate (px)	170	225	290
Erro de pontaria	≈ 12,5°	≈ 4,5°	≈ 1°
Tempo de reação da IA (s)	0,35	0,20	0,08
Guarda-redes: velocidade (px/s)	170	230	280
Guarda-redes: tempo de reação (s)	0,22	0,16	0,11
Guarda-redes: alcance (px)	16	17	18
Guarda-redes: antecipação do remate	52 %	60 %	70 %

A dificuldade só se aplica à equipa adversária; os colegas e o guarda-redes do jogador humano ficam sempre no nível normal.

Guarda-redes. O guarda-redes só se move na vertical, sobre a linha de golo. Não é perfeito: guarda as últimas posições da bola e reage ao que viu há alguns décimos de segundo (tempo de reação). Quando há um remate, calcula onde a bola vai cruzar a linha de golo, y_cruz = y_bola + v_y × (x_GR − x_bola) / v_x, mas só antecipa parte do caminho:

vai para y_bola + (y_cruz − y_bola) × antecipação. Assim um remate feito direto para ele é sempre defendido e só um remate bem colocado no canto pode entrar.

Defesas sem falhas. Como a bola anda cerca de 10 px por frame, não basta ver onde ela está: podia «saltar» por cima do guarda-redes entre dois frames. Por isso o jogo calcula o ponto do caminho da bola nesse frame (o segmento entre a posição anterior e a atual) mais perto do guarda-redes e compara essa distância com o alcance dele. Esta verificação é feita antes de ver se é golo, e uma defesa manda sempre a bola para a frente da baliza. Os jogadores de campo também chocam com o guarda-redes (são afastados a 30 px dele) e a IA nunca escolhe uma posição tática na pequena zona à frente da baliza.

Ordem de desenho. A bola entra na mesma ordenação por profundidade que os jogadores: se passar atrás do guarda-redes ou de um jogador, fica tapada por ele. Antes era desenhada sempre por cima de todos, o que fazia parecer que atravessava o corpo do guarda-redes.

Câmara em perspetiva. A física continua num campo plano visto de cima. A câmara só muda o desenho: o campo passa a ser um trapézio, mais estreito ao fundo. Para uma posição t (0 = fundo, 1 = frente), a escala horizontal é $f(t) = f_{topo} + (f_{base} - f_{topo})\cdot t$ e a posição vertical no ecrã é proporcional ao integral $G(t) = f_{topo}\cdot t + (f_{base} - f_{topo})\cdot t^2/2$. Para saber onde o rato aponta no campo é preciso fazer a conta ao contrário, o que dá uma equação do 2.º grau, resolvida com a fórmula resolvente. Os jogadores do fundo são desenhados mais pequenos e antes dos da frente (ordenados pela posição y), para ficarem por trás.

Equipamentos das equipas. Os sprites dos jogadores são desenhados com um equipamento azul. Ao carregar a partida, os píxeis com tom (hue) entre 180° e 260° são pintados com as cores do clube, mantendo as sombras. As imagens recoloridas ficam guardadas em cache para não repetir o trabalho.

Animações. Cada jogador tem um controlador de animações com estados (parado, correr, com bola, passe, remate, queda). A velocidade da animação acompanha a velocidade do jogador, e há uma inclinação para a frente ao patinar e um pequeno efeito ao mudar de direção.

Modo carreira. O jogador escolhe a equipa e a dificuldade dos adversários (Figura 10) e joga um campeonato contra as outras 7 equipas, uma vez contra cada (7 jornadas de 4 jogos). Em cada jornada joga o seu jogo ou carrega em «Simular» (também pode simular a época toda); os outros 3 jogos são sempre simulados. Vitória vale 3 pontos, empate 1 e derrota 0. O código está em carreira.py (dados e regras) e carreira_menu.py (ecrãs).

Calendário. Usa-se o método do círculo: as 8 equipas são postas numa roda, a primeira fica fixa e as outras rodam uma posição em cada jornada; em cada jornada joga a 1.ª contra a 8.ª, a 2.ª contra a 7.ª, etc. Assim cada equipa joga exatamente uma vez contra cada uma das outras (28 jogos). Para decidir quem joga em casa, fica em casa quem tem menos jogos em casa até ali (e, em caso de empate, quem jogou fora na jornada anterior); assim cada equipa tem 3 ou 4 jogos em casa. A ordem das equipas é baralhada em cada época, para o calendário mudar.

Jogos simulados. Cada equipa tem uma força entre 0 e 100 (do FC Porto, 90, ao CRP Lavra, 67). Na primeira versão os golos seguiam uma distribuição de Poisson, mas apareciam resultados sem sentido, como 14-3, e a equipa muito mais fraca ganhava em 11,5 % dos jogos. O modelo novo faz três coisas: (1) soma à força de cada equipa a «forma do dia», um valor aleatório com distribuição normal (desvio padrão 4,5), e a forma recente (+1,5 por cada vitória e −1,5 por cada derrota nos últimos 3 jogos); (2) calcula os golos esperados com golos = 3,1 × e^(0,24 × diferença de forças / 10), mais 0,3 para quem joga em casa; (3) sorteia os golos com uma distribuição binomial (várias oportunidades, cada uma com 50 % de dar golo), que tem a mesma média mas varia menos, e com um máximo de 11. Quando é o jogador a simular o seu jogo, a dificuldade também conta (+5 de força no Fácil, −5 no Difícil). A Tabela 8 mostra o efeito.

Probabilidades e forma. O cartão do próximo jogo mostra as probabilidades de vitória, empate e derrota se o jogo for simulado. São calculadas pelo método de Monte Carlo: o jogo é simulado 600 vezes com o mesmo modelo e conta-se quantas vezes cada resultado acontece. A tabela mostra também a forma das equipas (os últimos 5 resultados como bolinhas verdes, amarelas e vermelhas) e os jogos simulados do jogador têm a etiqueta «SIM» (coluna simulado da tabela Partida).

Planteis. Cada clube tem o seu plantel real, definido em team.py: um guarda-redes e quatro jogadores de campo, como no hóquei em patins (antes o jogo tinha cinco jogadores de campo com nomes inventados e iguais para todos os clubes). Os quatro jogam em losango: um avançado, dois médios nas alas e um defesa. Os nomes aparecem no cartão do jogador ativo, nas faltas e nos golos. Ao arrancar, o modo carreira copia os planteis para a tabela Jogador com um «upsert» (INSERT ... ON CONFLICT DO UPDATE): se um nome mudar em team.py, a base de dados atualiza-se sozinha sem perder os golos das épocas anteriores.

Marcadores. Num jogo jogado, a bola guarda o último jogador de cada equipa que lhe tocou (ao ficar com ela ou num desvio). Quando há golo, o marcador é o último jogador da equipa que marcou a tocar na bola, e o golo fica registado com o minuto. No fim do jogo aparecem os marcadores de cada equipa e, no modo carreira, são gravados na tabela Estatistica. Nos jogos simulados, cada golo é sorteado entre os jogadores de campo, com mais hipóteses para o avançado (40 %) do que para os médios (23 % cada) e o defesa (14 %). A tabela dos melhores marcadores é uma consulta com SUM, GROUP BY e ORDER BY.

Força das equipas no jogo. A força dos clubes também conta nos jogos jogados: a velocidade dos jogadores da IA é multiplicada por 1 + (força − 79) / 23 × 0,10 (cerca de +5 % no FC Porto e −5 % no CRP Lavra) e a pontaria melhora na mesma proporção. A dificuldade continua a ser o que mais conta.

Base de dados. A classificação é lida da base de dados com ORDER BY pontos DESC, diferença de golos DESC, golos marcados DESC. Gravar um resultado (o jogo do jogador, os 3 simulados e a atualização da classificação) é feito numa só transação: se alguma coisa falhar, nada fica gravado e o que estava antes não se perde. Todas as consultas

usam parâmetros (?), nunca texto colado na SQL, e a base de dados recusa golos negativos (CHECK golos >= 0).



Figura 10 – Modo carreira: escolha da equipa e da dificuldade



Figura 11 – Modo carreira: classificação com a forma, próximo jogo com as probabilidades e resultados da jornada

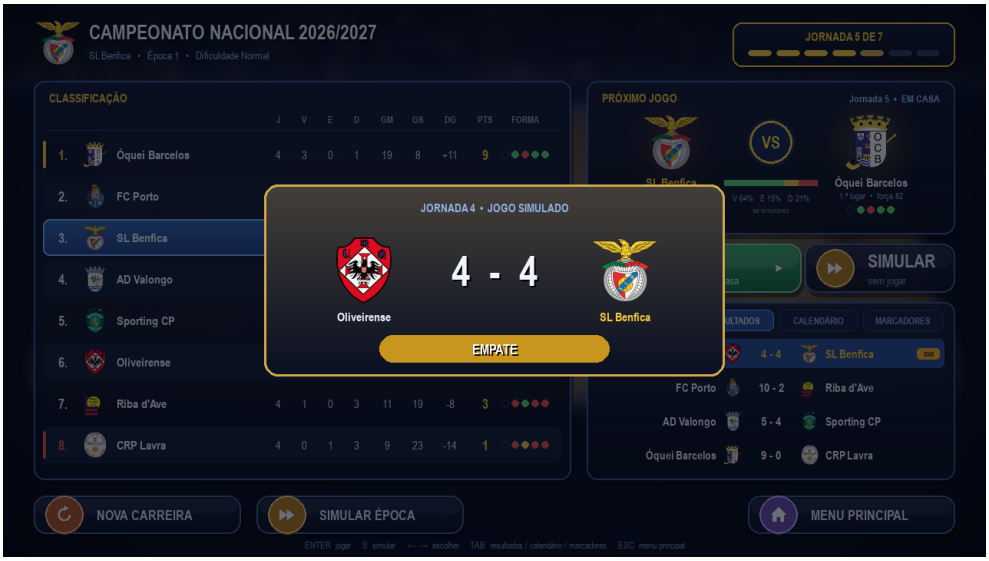


Figura 12 – Modo carreira: resultado de um jogo simulado



Figura 13 – Modo carreira: fim da época, com o campeão e os melhores marcadores

Menu principal e definições. O menu principal tem só três botões grandes (Jogo rápido, Modo carreira e Sair), cada um com um ícone e uma frase a explicar o que faz. Os controlos saíram da lista e passaram para uma engrenagem no canto superior direito, que abre o ecrã de Definições: à esquerda a lista de teclas e à direita as opções de ecrã inteiro (também com F11), estilo arcade e volume do som, que ficam gravadas num pequeno ficheiro JSON. O fundo é o próprio pavilhão desfocado, com dois projetores a varrer o ecrã e partículas de pó na luz. O título tem um gradiente, contorno e um reflexo que passa de vez em quando. Os ícones (engrenagem, troféu, «desligar») e o emblema dos sticks cruzados são desenhados no código, no módulo graficos.py. Os botões mudam de cor aos poucos (transição suave) quando recebem o foco, com o rato ou com as setas.

Gráficos. Para melhorar a qualidade gráfica sem tornar o jogo mais lento, quase tudo o que é novo é desenhado uma só vez e guardado em cache. O piso passou a ser desenhado com o dobro da resolução e só depois é reduzido (super-amostragem), o que

deixa as linhas e o emblema do centro suaves, sem «escadinhas». Cada tábua de madeira tem um tom diferente, veios e alguns nós; junto às tabelas o piso fica mais escuro; e as luzes do pavilhão refletem-se no verniz (manchas de luz somadas às cores). Os jogadores, os guarda-redes e a bola têm sombras com a borda esbatida. As animações dos jogadores não foram alteradas. Com vsync, a imagem fica sincronizada com o monitor.

Sons. Não foram usados ficheiros de som de fora: os 14 sons são criados por síntese no módulo sons.py, a partir de ondas sinusoidais e ruído aleatório, e gravados em ficheiros WAV na primeira vez que o jogo arranca. O apito é um tom de cerca de 3 kHz com o «trinado» da bolinha (modulação a 31 Hz); as tacadas são um estalo curto com duas frequências de madeira que se apagam depressa; a pancada na tabela é uma frequência grave que desce; o poste soa a metal porque tem várias frequências que não são múltiplas umas das outras; e o público é ruído filtrado nas frequências da voz humana (filtros passa-banda), com subidas e descidas lentas. No golo toca uma buzina (acorde com vibrato) e o público a festejar.

Sons no jogo. A bola não sabe nada de som: quando é rematada, bate na tabela, no poste ou no guarda-redes, guarda um «acontecimento» numa lista, e o ciclo do jogo transforma cada um num som, com o volume conforme a força e mais à esquerda ou à direita conforme a posição no ecrã. Um som igual não se repete em menos de 70 ms. O público ouve-se baixo nos menus e mais alto durante a partida, e há um «uhhh» quando a bola bate no poste ou há uma grande defesa. Nas Definições há o volume dos efeitos e do público, gravado com o ecrã inteiro. Se o computador não tiver placa de som, o jogo continua sem som.

Estilo arcade. Inspirado no estilo dos jogos desportivos arcade (como o Tape to Tape), mas sem copiar imagens, personagens ou regras, foi criado o módulo efeitos.py, que dá mais energia ao jogo sem mexer na física nem na IA. Nos impactos (remate forte, defesa, roubo de bola, bola no poste) o jogo congela durante 40 a 70 ms («hit-stop»), o ecrã treme, saltam faíscas e aparece um texto de banda desenhada («DEFESA!», «POSTE!», «ROUBO!»), que entra a crescer e sobe a desaparecer. A bola deixa um rasto quando vai depressa e as tabelas levantam pó. Os jogadores têm um contorno escuro (a forma do sprite, obtida com uma máscara de píxeis, é desenhada em escuro deslocada em 8 direções). No golo aparece um «GOLO!» gigante numa explosão em estrela na cor da equipa, com o nome de quem marcou, um clarão, confetes e câmara lenta durante 2 segundos, e só depois a bola volta ao centro.

Tempo de jogo e ecrã a tremer. O relógio do jogo passa pelo gestor de efeitos: em cada frame recebe o tempo real e devolve o tempo que o jogo deve andar (0 durante uma pausa, 25 % durante a câmara lenta, voltando aos poucos ao normal). O tremer só desloca a imagem do campo; o marcador e o painel de baixo ficam parados para se lerem bem. O estilo arcade liga-se e desliga-se nas Definições e a escolha fica gravada; com ele desligado o jogo fica exatamente como antes.

Remate na primeira pessoa. Antes, o remate era uma seta de mira no campo visto de cima. Agora, ao carregar em X com a bola, o jogo para e a câmara passa para os olhos do jogador, virada para a baliza, com o guarda-redes, a rede, as tabelas e o público. O jogador

aponta com o rato ou com as setas a qualquer ponto da baliza, também em altura, e mantém X carregado para encher a barra de força (que sobe e desce, por isso é preciso largar no momento certo). Mais força faz a bola chegar mais depressa, mas aumenta o erro: o círculo da mira cresce com força³. Num jogo há 5 segundos para rematar (os adversários estão a chegar); no fim do tempo o remate sai sozinho. O código está em remate_pov.py.

Perspetiva na primeira pessoa. Cada ponto do campo (x, y no chão e altura h) é convertido para o ecrã com uma projeção em perspetiva: z = distância à frente da câmara e x = distância para o lado (produtos escalares com as direções «em frente» e «direita»); ecrã_x = centro + F·x/z e ecrã_y = horizonte + F·(olhos − h)/z, com F = 1000. O que fica atrás da câmara é cortado (algoritmo de Sutherland–Hodgman para os polígonos do piso e das tabelas). Para a mira seguir o rato faz-se a conta ao contrário: a partir da posição do rato calcula-se o ponto da baliza (u, h). Como a câmara não se mexe durante o remate, o fundo, o piso, as linhas e a rede são desenhados uma só vez; assim cada frame demora só 2 a 4 ms.

Guarda-redes no remate. O resultado é decidido por uma função à parte (decidir_remate), que se pode testar sem desenho: o erro do remate desvia o ponto apontado; o tempo de voo é distância / velocidade; o guarda-redes inclina-se um pouco para onde o jogador aponta, só se mexe depois do tempo de reação, mergulha com uma velocidade máxima e «lê» o lado do remate com uma probabilidade que diminui nos remates rápidos. O alcance dele é maior em baixo (pernas) do que nos cantos de cima. Se a bola passar fora dos postes ou por cima é «fora», perto dos ferros é «poste», ao alcance do guarda-redes é «defesa», e senão é golo. Os valores dependem da dificuldade e foram afinados simulando milhares de remates (Tabela 9). Depois da animação, o jogo 2D continua com a bola no sítio certo: na rede, no ressalto do guarda-redes, do poste, ou atrás da baliza.

Modo teste. No menu principal há a opção MODO TESTE: só o jogador e o guarda-redes adversário, sem tempo e sem limite de remates. Depois de cada remate o jogador volta a ter a bola num sítio diferente à frente da baliza (ou carrega em R). O marcador mostra golos / remates e a pausa mostra as defesas e os remates para fora. Serve para treinar e para testar o remate e a dificuldade do guarda-redes (F4).



Figura 14 – Remate na primeira pessoa: mira, círculo de erro e barra de força



Figura 15 – Estilo arcade: festa de um golo, com o nome de quem marcou

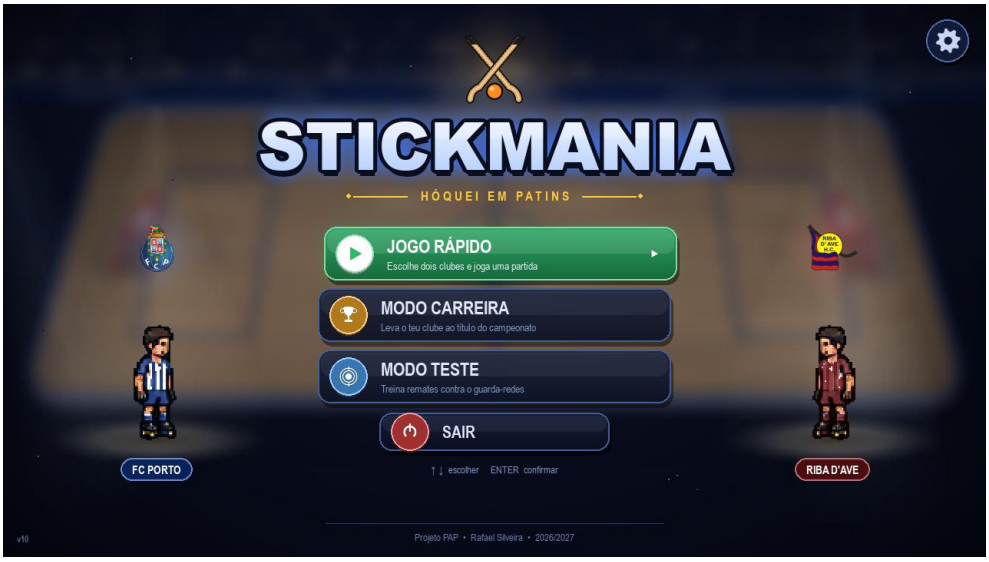


Figura 16 – Menu principal novo, com a engrenagem das definições no canto

3.3 Integração hardware – software

Neste projeto, a integração é entre o jogo e os periféricos do computador. A entrada é lida com os eventos do pygame (teclas premidas e cliques) e com o estado do teclado em cada frame (para o movimento contínuo). A saída é desenhada numa superfície de 1280×720 que é mostrada no ecrã 60 vezes por segundo.

Tabela 4 – Controlos do jogo

Tecla / botão	Ação
W A S D	Mover o jogador ativo
Shift	Sprint (patinar mais depressa)
Z	Passar a bola
X	Remate na primeira pessoa: o rato ou as setas apontam à baliza; manter X (ou o botão do rato) enche a barra de força e largar remata
TAB / C / Espaço	Trocar para o jogador mais perto da bola
ESC / P	Pausa (no ecrã do remate, ESC cancela o remate)
R	No MODO TESTE: novo remate noutro sítio
T / Y	Trocar a equipa da casa / de fora
F4	Mudar a dificuldade durante o jogo
F11	Ecrã inteiro (também nas Definições, na engrenagem do menu)
F3	Mostrar informação de depuração

3.4 Decisões técnicas e dificuldades ultrapassadas

Perspetiva sem mudar a física. A primeira ideia foi transformar toda a física para 3D. Em vez disso, a física ficou num campo plano e só o desenho passou pela câmara. Assim a IA e as colisões não mudaram e não apareceram erros novos.

Balizas desenhadas no código. Os PNG antigos das balizas estavam desenhados de lado e não combinavam com o rinque. As balizas passaram a ser desenhadas no código a partir de pontos 3D projetados pela câmara, em duas camadas: a rede de trás (desenhada antes dos jogadores) e os postes da frente (desenhados depois). Assim a bola e o guarda-redes parecem estar dentro da baliza.

Desempenho. Desenhar o rinque em perspetiva linha a linha é lento. Por isso, o rinque, as tabelas e as bancadas são desenhados uma só vez, quando a partida começa, e depois só se copia essa imagem em cada frame. O jogo arranca em menos de um segundo e mantém os 60 FPS.

Pontaria com o rato. Com a câmara, a posição do rato no ecrã já não é a posição no campo. Foi preciso escrever a função inversa da câmara (desprojetar), com a equação do 2.º grau descrita na secção 3.2.

Sprites. Foi criada uma folha de sprites nova com mais animações, mas o resultado visual ficou pior do que o original e a alteração foi revertida com o Git. A melhoria dos sprites fica para uma fase seguinte.

Bola presa e jogadores colados. Ao início a bola atravessava as tabelas quando ia muito rápida e os jogadores ficavam presos uns nos outros. Foi corrigido limitando a velocidade, verificando a posição depois de cada movimento e separando os jogadores que se sobrepõem.

Colisões e IA com comportamentos estranhos. Mesmo depois disso, os jogadores ainda «tremiam» quando iam uns contra os outros, a bola mudava de dono várias vezes por segundo e a IA tinha comportamentos estranhos: os avançados não voltavam para defender, ninguém ia a uma bola parada entre dois colegas, e quase não havia golos porque o guarda-redes tapava a baliza toda e reagia instantaneamente. Para resolver isto sem «adivinhar», foi criado um script que joga partidas IA contra IA sem janela e conta os problemas (Tabela 6). Cada correção foi medida antes e depois. Também se descobriu que, depois de qualquer golo, a bola ia sempre para a equipa do jogador (agora vai para quem sofreu o golo) e que a dificuldade «Difícil» também melhorava os colegas do jogador.

Bola e guarda-redes. Ao jogar, ainda se viam problemas com a bola e o guarda-redes. Foi feito um segundo script que procura situações estranhas (Tabela 7): a bola saltava até 50 px de um frame para o outro quando o jogador mudava de direção; jogadores ficavam em cima do guarda-redes porque não chocavam com ele; a bola parecia atravessar o guarda-redes por ser sempre desenhada por cima; e, em casos raros, a bola entrava nos cantos arredondados ou ficava no stick dentro da rede. As soluções estão descritas na secção 3.2.

Porquê SQLite. Foi escolhida uma base de dados SQLite (e não um ficheiro de texto ou JSON) porque já vem com o Python, não precisa de servidor, garante as transações (o campeonato nunca fica meio gravado) e permite ordenar a classificação com uma consulta SQL. Também segue os requisitos 17 a 19 e o modelo de dados do relatório.

Regras do modo carreira. No modo carreira o jogador controla sempre a sua equipa, quer jogue em casa quer fora; nos jogos fora o rinque passa a ter o emblema e as cores do adversário. Se o jogador desistir a meio (na pausa), o jogo não conta e pode ser jogado outra vez; não há «reiniciar» nem troca de equipa ou de dificuldade durante o jogo, para o resultado ser justo.

Porquê sons sintetizados. Usar sons descarregados da Internet levantava problemas de direitos de autor e de qualidade diferente entre eles. Criar os sons no código garante que são todos originais, que o projeto não depende de ficheiros de fora e permite mostrar na defesa como funcionam (ondas, ruído e filtros).

Quatro jogadores de campo. Com quatro jogadores de campo (as regras reais) o jogo ficou com menos golos: entre duas equipas da IA passou de 3,0 para 0,6 golos em 3 minutos, porque há menos ressaltos e o guarda-redes vê todos os remates. Medindo com o script da Tabela 6, foram feitos três ajustes: os médios e o defesa sobem mais no ataque, a IA remata de um pouco mais longe (255 px no nível normal, antes 225) e a antecipação

do guarda-redes no nível normal passou de 60 % para 56 %. O resultado foi 1,8 golos em 3 minutos, cerca de 6 por jogo, e menos bolas a trocar de dono várias vezes seguidas (de 18 para 6).

Afinar a simulação com dados. Para escolher os valores do modelo de simulação não se «adivinhou»: foi criado o script ferramentas/analise_simulacao.py, que simula 600 épocas e mede golos por jogo, empates, goleadas, surpresas e quem é campeão. Os parâmetros foram ajustados até os resultados parecerem os de um campeonato real.

4. Testes e validação

4.1 Plano de testes

Os testes foram de dois tipos: testes manuais (jogar e verificar cada funcionalidade) e testes automáticos, com scripts que arrancam o jogo sem janela (SDL_VIDEODRIVER=dummy), simulam teclas e verificam o resultado. A tabela resume o plano de testes.

Tabela 5 – Plano de testes

N.º	Teste	Resultado esperado	Resultado
T01	Arrancar o jogo com jogar.bat	Abre o menu principal	Passou
T02	Escolher equipas e dificuldade e começar	A partida começa com as equipas escolhidas	Passou
T03	Mover o jogador com WASD e Shift	Move-se nas 8 direções e acelera com Shift	Passou
T04	Passar (Z) e rematar (X)	A bola sai na direção escolhida	Passou
T05	Bola contra as tabelas a alta velocidade	Ressalta e nunca sai do rinque	Passou
T06	Bola a entrar na baliza	Golo somado à equipa certa e bola ao centro	Passou
T07	Fim da 1.ª e da 2.ª parte	Intervalo e depois ecrã de resultado final	Passou
T08	Pausar e continuar	Marcador e tempo mantêm-se	Passou
T09	Trocar equipas durante o jogo (T / Y)	Símbolos, cores e equipamentos mudam	Passou
T10	Rato em perspetiva (projetar e desprojetar)	Erro de 0 px na conversão ida e volta	Passou
T11	Desempenho	60 FPS estáveis durante a partida	Passou
T12	Jogadores a chocar uns com os outros	Separam-se sem tremer nem ficar sobrepostos	Passou
T13	Disputa de bola entre dois jogadores	A bola não muda de dono várias vezes seguidas	Passou
T14	IA defende quando o adversário tem a bola	O mais perto pressiona e outro faz cobertura	Passou
T15	Golo marcado	Recomeça com a bola a equipa que sofreu o golo	Passou
T16	Mudar a dificuldade	Só muda a equipa adversária	Passou

N.º	Teste	Resultado esperado	Resultado
T17	Jogador muda de direção com a bola	A bola roda à volta dele, sem saltos	Passou
T18	Remate rápido junto ao guarda-redes	Não atravessa o GR; a defesa manda a bola para a frente	Passou
T19	Jogador a ir contra o guarda-redes	Não fica em cima do GR	Passou
T20	Bola nos cantos do rinque	Ressalta na curva da tabela	Passou
T21	Calendário do campeonato	7 jornadas, 28 jogos, cada par joga 1 vez, 3 ou 4 jogos em casa	Passou
T22	Época completa no modo carreira	Pontos = 3×V + E; golos marcados = sofridos no total	Passou
T23	Fechar e voltar a abrir o jogo	O campeonato continua no mesmo ponto	Passou
T24	Gravar golos negativos	Recusado e nada é gravado (transação)	Passou
T25	Desistir de um jogo na pausa	O jogo não conta e pode ser jogado outra vez	Passou
T26	Nova época / nova carreira	Época 2 com calendário novo; nova carreira pede confirmação	Passou
T27	Clicar na engrenagem / ↑ e ENTER no menu	Abre as Definições; ESC volta ao menu	Passou
T28	Ecrã inteiro (interruptor e F11)	Muda para ecrã inteiro e volta; a opção fica gravada	Passou
T29	Tempo de desenho de cada frame	Menu 7,6 ms e partida 5,4 ms (menos de 16,7 ms = 60 FPS)	Passou
T30	Sons durante uma partida automática	Apitos, passes, remates, defesas e público a tocar	Passou
T31	Volume dos efeitos e do público	Muda com ← → e fica gravado	Passou
T32	Simular o jogo do jogador (tecla S ou botão)	Resultado gravado com simulado = 1; tabela certa	Passou
T33	Simular a época	Pede confirmação; joga as jornadas que faltam	Passou
T34	Abrir uma gravação da versão anterior	A coluna simulado é acrescentada; nada se perde	Passou
T35	Modelo de simulação (600 épocas)	Resultados realistas (Tabela 8)	Passou
T36	Desenho do ecrã do campeonato	4,7 ms por frame (antes da cache era 25 ms)	Passou
T37	Planteis de todos os clubes	4 jogadores + GR com os nomes certos; mudam ao trocar de equipa	Passou
T38	Golo num jogo jogado	Marcador e minuto certos na mensagem e no fim do jogo	Passou
T39	Marcadores no modo carreira	Soma dos golos dos jogadores = soma dos golos dos resultados	Passou
T40	Jogabilidade com 4 jogadores	1,8 golos e 33 remates em 3 min (IA contra IA)	Passou
T41	Efeitos arcade num jogo IA contra IA	Aparecem defesa, poste, roubo, rasto e golo	Passou
T42	Interruptor «Estilo arcade» nas Definições	Liga e desliga com teclado e rato; fica gravado	Passou
T43	Golo com e sem estilo arcade	Com: festa e câmara lenta; sem: faixa normal e reposição logo	Passou
T44	Desenho com os efeitos	4,7 ms por frame (95 % dos frames abaixo de 11,5 ms)	Passou
T45	X abre o remate na primeira pessoa	Mira com o rato no ponto certo (erro < 1,5 px)	Passou

N.º	Teste	Resultado esperado	Resultado
T46	Carregar e largar X	A barra enche; ao largar remata e mostra o resultado	Passou
T47	ESC no ecrã do remate	Cancela e o jogador continua com a bola	Passou
T48	Tempo limite num jogo (5 s)	O remate sai sozinho	Passou
T49	Remate atrás da linha de golo	Remate rápido sem abrir o ecrã	Passou
T50	MODO TESTE: 31 remates seguidos	7 golos, 18 defesas, 6 fora; volta sempre a ter a bola	Passou
T51	Desenho do ecrã do remate	2 ms a apontar e 3,7 ms no remate (antes 15–20 ms)	Passou

4.2 Ensaios, calibração e medições

Foram feitas medições com o jogo a correr sem limite de FPS, em simulação automática:

- **Tempo de arranque:** cerca de 0,75 s até a partida estar pronta (inclui desenhar o rinque em perspetiva).
- **Velocidade de desenho:** entre 250 e 290 frames por segundo sem limite, muito acima dos 60 FPS necessários.
- **Precisão da câmara:** converter um ponto do campo para o ecrã e de volta dá erro de 0,0 píxeis.
- **Golos:** em simulações com remates automáticos, todos os golos foram detetados e atribuídos à equipa certa.

Também foram afinados vários valores por tentativa e erro: o coeficiente de ressalto passou de 0,72 para 0,78 (a bola parecia «morta»), a velocidade de sprint passou de 330 para 260 px/s (era demasiado rápida), o tamanho da bola passou de 14 para 11 px e o atrito foi dividido em dois valores (bola rápida e bola lenta) para a bola parar de forma mais natural.

Para a jogabilidade foi criado um script de medição que põe as duas equipas controladas pela IA a jogar sem janela, 8 jogos de 3 minutos, sempre com as mesmas sementes aleatórias (para a comparação ser justa), e conta os problemas. A tabela compara a versão antiga com a versão corrigida.

Tabela 6 – Medição da jogabilidade antes e depois das correções (média por jogo de 3 min, IA contra IA)

Medida	Antes	Depois
Golos	0,4	2,5
Remates	20	51
Bola roubada e devolvida em menos de 0,6 s («pingue-pongue»)	101	18
Passes (muitos eram intercetados)	378	81
Faltas apitadas	15,1	0,6
Frames com jogadores a «tremer» (inverter o sentido)	59	39
Momentos com jogadores sobrepostos	4,8	0,2

Medida	Antes	Depois
Momentos com jogadores fora do rinque	0,4	0

Um segundo script procura problemas da bola e do guarda-redes, em 4 jogos de 3 minutos (IA contra IA):

Tabela 7 – Bola e guarda-redes antes e depois das correções (média por jogo de 3 min)

Medida	Antes	Depois
Saltos da bola de mais de 25 px num frame	157	0
Frames com um jogador em cima do guarda-redes	107	3,5
Frames com a bola fora dos cantos arredondados	0,75	0
Frames com a bola no stick dentro da rede	4	0
Bola a passar junto ao GR sem defesa (a 22–24 px, fora do alcance)	1,0	0,5

Para o modo carreira, o script ferramentas/analise_simulacao.py simulou 600 épocas completas com o modelo antigo (Poisson) e com o novo:

Tabela 8 – Modelo de simulação dos jogos: antigo e novo (600 épocas, 16 800 jogos)

Medida	Antigo	Novo
Golos por jogo	6,6	6,9
Empates	12,9 %	12,8 %
Surpresas (a equipa 12+ pontos mais fraca ganha)	11,5 %	6,3 %
Goleadas (6 ou mais golos de diferença)	8,4 %	7,2 %
Maior número de golos de uma equipa	18	11
FC Porto (força 90) campeão	38,6 %	43,8 %
CRP Lavra (força 67): posição média	7,30	7,33

Para o remate na primeira pessoa simularam-se 1200 remates para cada caso. A primeira versão do modelo era desequilibrada: a meia distância nem os remates aos cantos passavam dos 20 % e com muita força a bola ia para fora em até 64 % das vezes; de perto, os cantos davam 95 % de golos. Depois de afinar a leitura do guarda-redes, a velocidade de mergulho e o erro do remate, os resultados no nível normal (força de 65 %) ficaram assim:

Tabela 9 – Remate na primeira pessoa: percentagem de golos no nível normal (força 65 %)

Distância	Meio	Canto de baixo	Canto de cima	Lado, meia altura
5,3 m	0 %	36 %	90 %	35 %
8,8 m	6 %	29 %	33 %	36 %
13,3 m	7 %	17 %	20 %	23 %

Rematar para o meio quase nunca dá golo; os cantos compensam, sobretudo os de cima de perto; e de longe é mais difícil. No MODO TESTE, 31 remates seguidos deram 7 golos (23 %).

4.3 Resultados obtidos e ajustes

Todos os testes (T01 a T51) passaram. O modo carreira foi testado com um script que joga uma época inteira como se fosse um jogador (carregando nas teclas), com uma base de dados temporária, e verifica a classificação, a gravação e a passagem entre os ecrãs.

Os sons não se podem ouvir num teste automático, por isso o teste T30 «espia» o gestor de som: conta que sons foram pedidos durante 2 minutos de jogo IA contra IA (por exemplo, 22 passes, 12 remates, 4 defesas e os apitos de início, faltas, intervalo e fim). Os testes correm com o controlador de áudio «dummy», que não faz barulho.

O primeiro calendário tinha os jogos em casa mal distribuídos (uma equipa nunca jogava em casa); foi corrigido escolhendo para casa a equipa com menos jogos em casa.

Os testes levaram a vários ajustes: o triângulo de seleção e a etiqueta «GR» ficavam a flutuar acima dos jogadores depois da câmara em perspetiva e passaram a usar o retângulo visível do sprite; o stick ficava demasiado claro e foi escurecido; as tabelas e as balizas foram redesenhadas para ficarem iguais à imagem de referência; e a IA foi afinada para haver menos quedas.

Na afinação do guarda-redes testaram-se várias combinações de alcance, antecipação e tempo de reação. Com o alcance antigo (34 px) o guarda-redes tapava a baliza toda e quase não havia golos; com antecipação perfeita da trajetória também não. Uma primeira versão com um erro de leitura aleatório fazia o guarda-redes mexer-se às vezes para o lado errado num remate direto, o que parecia um erro, e foi trocada pela antecipação parcial. O número de golos muda muito com este valor (com 55 % havia 6 golos em 3 minutos, com 62 % só 1), por isso escolheu-se 60 % no nível normal: cerca de 5 % dos remates dão golo entre duas equipas da IA. Com a passagem a quatro jogadores de campo este valor foi baixado para 56 % (ver a secção 3.4).

5. Gestão do projeto

5.1 Planeamento e cronograma ◆ MADS

A tabela seguinte mostra o cronograma do projeto, desde o anteprojeto até à defesa.

Tabela 10 – Cronograma do projeto

Período	Tarefa	Estado
set.–out. 2026	Anteprojeto, levantamento de requisitos, escolha das tecnologias	Concluído

Período	Tarefa	Estado
out. 2026	Protótipo: rinque, jogadores, bola e física básica	Concluído
out. 2026	Interface do mockup, símbolos dos clubes e seleção de equipas	Concluído
out. 2026	Mira de remate, IA com dificuldades, faltas	Concluído
out. 2026	Animações, balizas, câmara em perspetiva e bancadas	Concluído
out. 2026	Correção das colisões, nova IA tática e guarda-redes com reação	Concluído
out. 2026	Correção da bola no stick, das defesas do guarda-redes e da ordem de desenho	Concluído
nov.–dez. 2026	Estatísticas da partida, modo de 2 jogadores	Planeado
nov. 2026	Modo carreira: campeonato com classificação guardado em SQLite	Concluído
nov. 2026	Menu principal novo, definições, ecrã inteiro e melhoria gráfica	Concluído
nov. 2026	Sons, simulação realista dos jogos e novos ecrãs do modo carreira	Concluído
nov. 2026	Planteis reais (4 + GR), marcadores e melhores marcadores	Concluído
nov. 2026	Estilo arcade (efeitos nos golos e impactos)	Concluído
nov. 2026	Remate na primeira pessoa e MODO TESTE	Concluído
jan.–mar. 2027	Mais estatísticas (remates, assistências) e transferências	Planeado
mar.–abr. 2027	Testes com colegas, correção de erros, novos sprites	Planeado
abr.–mai. 2027	Relatório final, manual de utilizador e apresentação	Em curso
jun. 2027	Defesa da PAP	Planeado

5.2 Gestão de riscos ◆ MADS

Tabela 11 – Gestão de riscos

Risco	Prob.	Impacto	Medida
Perder código (avaria, ficheiro apagado)	Média	Alto	Tudo no GitHub; commits frequentes
Uma alteração estragar o que funcionava	Alta	Médio	Testar antes de cada commit; reverter com o Git
Atrasos no cronograma	Média	Alto	Versões pequenas e jogáveis; prioridades definidas
Jogo lento em computadores fracos	Baixa	Médio	Pré-desenhar o rinque; medir os FPS
Erros de física (bola fora do rinque)	Média	Alto	Testes automáticos de colisões e golos
Uso dos símbolos dos clubes	Baixa	Médio	Uso apenas educativo, sem fins comerciais
Ausência de um elemento do grupo	Baixa	Médio	Código modular e documentado; repositório partilhado

5.3 Controlo de versões e trabalho em equipa (Git/GitHub)

O projeto usa o Git para o controlo de versões, com o repositório no GitHub (github.com/RafaelSilveira23/Stickmaniapap). Cada conjunto de alterações é guardado num commit com uma mensagem a explicar o que mudou, por exemplo «feat: níveis de dificuldade da IA, redução de quedas e sistema completo de faltas». Assim é possível ver o histórico, comparar versões e voltar atrás, como aconteceu com a folha de sprites que foi revertida.

O código está dividido em módulos (física, IA, interface, câmara, rinque), o que permite a cada elemento do grupo trabalhar numa parte sem estragar as outras. Os diagramas também estão no repositório (pasta docs/diagramas), em formato draw.io, para poderem ser editados.

6. Conclusão

O StickMania cumpre o objetivo principal: é um jogo de hóquei em patins jogável, fluido e com as equipas portuguesas. Tem física da bola com atrito e ressaltos, uma IA com papéis táticos e três níveis de dificuldade, guarda-redes, faltas, duas partes, menus completos, uma apresentação parecida com a televisão, com câmara em perspetiva, bancadas e balizas em 3D, sons criados por síntese e um modo carreira em que se joga (ou simula) um campeonato completo, guardado numa base de dados SQLite.

O projeto permitiu aplicar e aprofundar muitas matérias do curso: programação orientada a objetos, organização em módulos, matemática aplicada (vetores, equações do 2.º grau, perspetiva), algoritmos de decisão, testes e controlo de versões. As maiores dificuldades foram a física das colisões e a câmara em perspetiva, resolvidas separando a lógica do jogo da forma como é desenhado.

Como trabalho futuro estão previstos: música nos menus, mais estatísticas por jogador (remates, assistências, defesas do guarda-redes), transferências entre épocas, um modo de dois jogadores no mesmo teclado e sprites novos com mais animações.

7. Anexos

Anexo A – Ficheiros dos diagramas. Todos os diagramas deste relatório estão na pasta docs/diagramas do projeto, em formato .drawio (abrem e editam-se em app.diagrams.net) e em imagem .png:

- 01_casos_de_uso.drawio – diagrama de casos de uso
- 02_arquitetura.drawio – arquitetura em blocos
- 03_estados_jogo.drawio – máquina de estados do jogo
- 04_fluxograma_ia.drawio – fluxograma da IA
- 05_fluxograma_ciclo.drawio – fluxograma do ciclo principal
- 06_classes.drawio – diagrama de classes
- 07_modelo_dados.drawio – modelo de dados SQLite
- 08_fluxograma_carreira.drawio – fluxograma do modo carreira

Anexo B – Manual de utilizador (resumo). 1) Instalar o Python 3.10 ou mais recente. 2) Fazer duplo clique em jogar.bat (instala o pygame-ce na primeira vez). 3) No menu, escolher «Jogo rápido» (escolher as duas equipas e a dificuldade) ou «Modo carreira» (escolher a equipa e jogar o campeonato; o progresso fica gravado sozinho; em cada jornada pode jogar ou simular o jogo). 4) Jogar com os controlos da Tabela 4. Para rematar, carregar em X com a bola, apontar com o rato e manter X carregado para escolher a força. O MODO TESTE do menu serve para treinar remates. A engrenagem no canto do menu abre as definições (lista de controlos, ecrã inteiro, estilo arcade e volume do som).

Anexo C – Código-fonte: github.com/RafaelSilveira23/Stickmaniapap